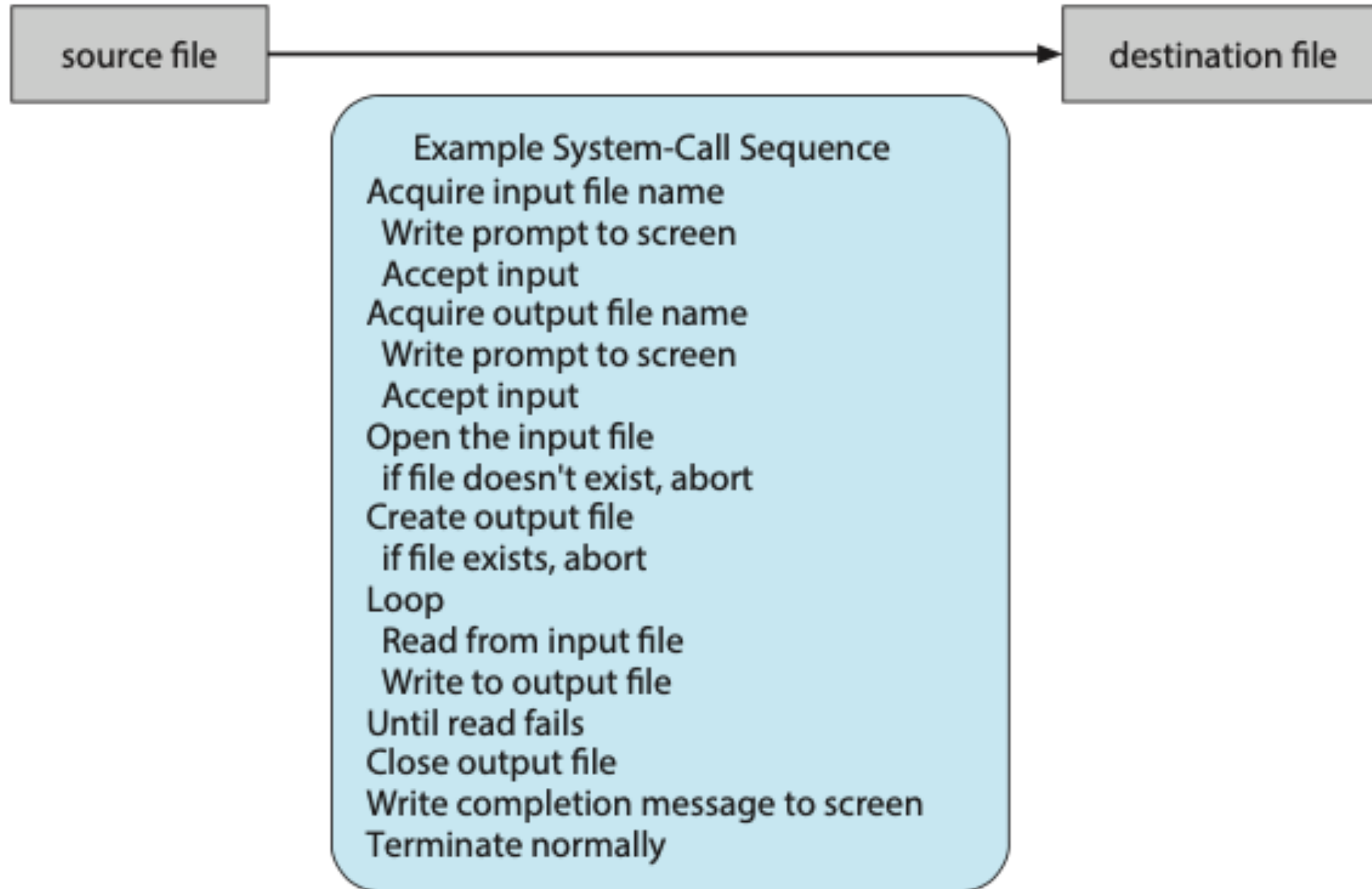

MODULE 2

Operating System Principles

SYSTEM CALLS

- ❑ Provide an interface to the services made available by an operating system.
- ❑ These calls are generally available as functions written in C and C++.
- ❑ It is a way for a user program to interface with the operating system.
- ❑ The program requests several services, and the OS responds by invoking a series of system calls to satisfy the request.
- ❑ A system call is a method for a computer program to request a service from the kernel of the operating system on which it is running.
- ❑ It acts as a link between the operating system and a process, allowing user-level programs to request operating system services.
- ❑ When a computer software needs to access the operating system's kernel, it makes a system call. The system call uses an API to expose the operating system's services to user programs. **It is the only method to access the kernel system.**

SYSTEM CALLS - EXAMPLE



SYSTEM CALLS – APPLICATION PROGRAMMING INTERFACE

- ❑ Application developers design programs according to an application programming interface (API).
- ❑ The API specifies a set of functions that are available to an application programmer, including the parameters that are passed to each function and the return values the programmer can expect.
- ❑ An API defines the correct way for a developer to request services from an operating system (OS) or other application and expose data within different contexts and across multiple channels.
- ❑ System calls offer the services of the operating system to the user programs via API (Application Programming Interface).

SYSTEM CALLS – APPLICATION PROGRAMMING INTERFACE

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t  read(int fd, void *buf, size_t count)
```

ssize_t	read	(int fd, void *buf, size_t count)
return value	function name	parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

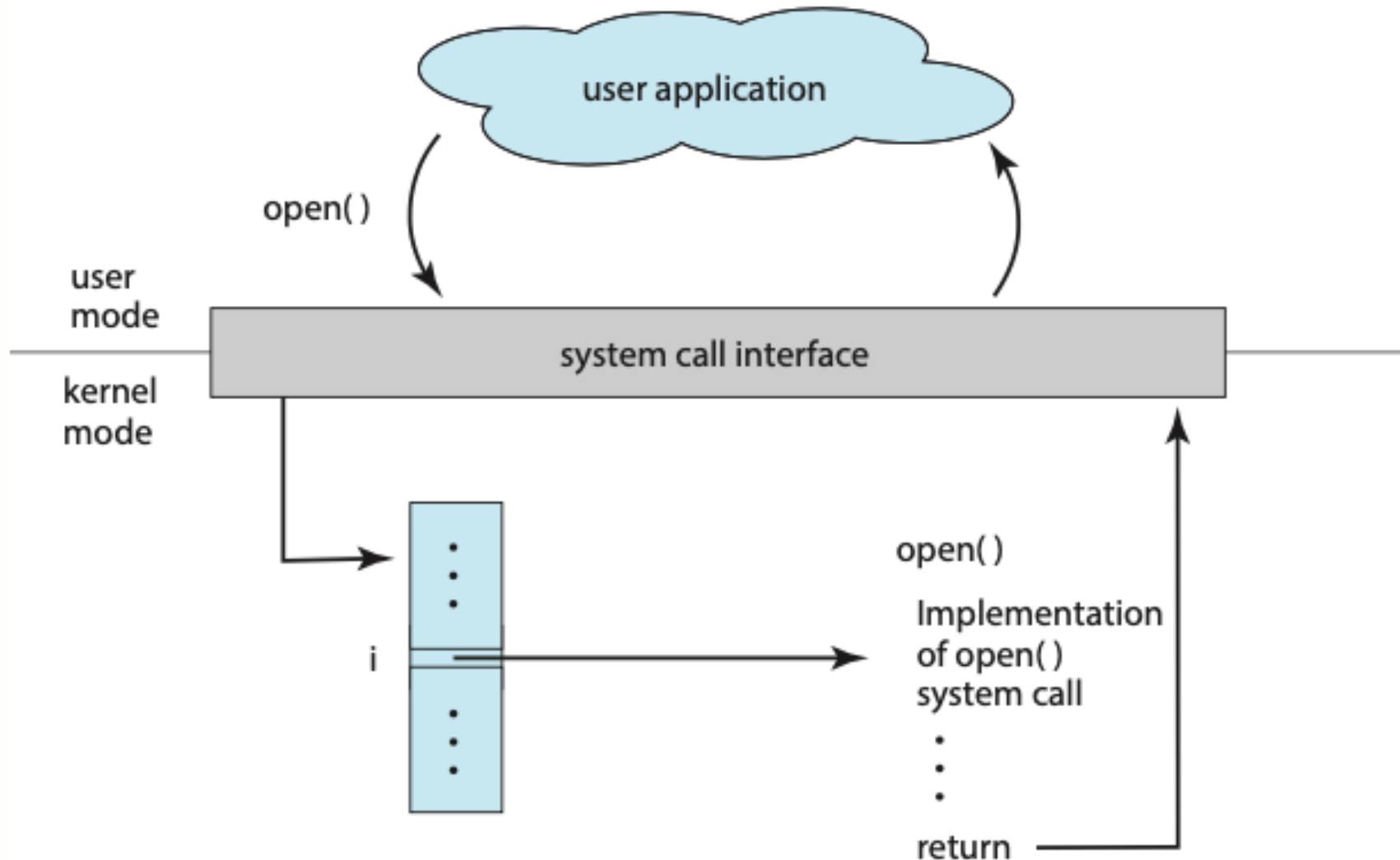
SYSTEM CALLS – APPLICATION PROGRAMMING INTERFACE

❑ **Benefits of API:**

- ❑ An application programmer designing a program using an API can expect his/her program to compile and run on any system that supports the same API .
- ❑ Actual system calls can often be more detailed and difficult to work with than the API available to an application programmer.

API – SYSTEM CALL – OS RELATIONSHIP

- ❑ The handling of a user application invoking the `open()` system call.



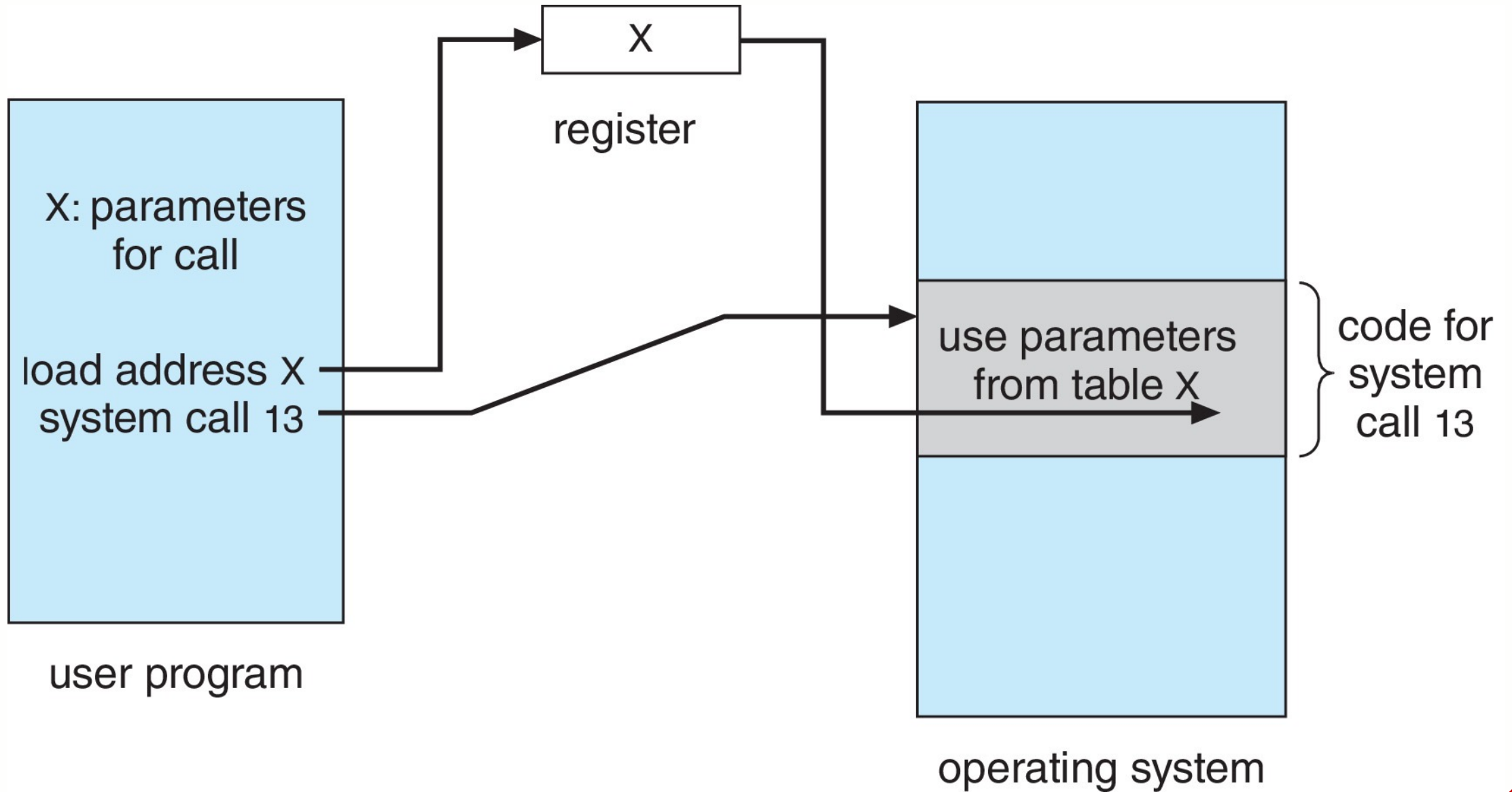
SYSTEM CALL IMPLEMENTATION

- ❑ A number is associated with each system call, and the system-call interface maintains a table indexed according to these numbers.
- ❑ The system-call interface then invokes the intended system call in the operating-system kernel and returns the status of the system call.
- ❑ The caller need know nothing about how the system call is implemented
 - ❑ Just needs to obey API and understand what OS will do as a result call
 - ❑ Most details of OS interface hidden from programmer by API
 - ❑ Managed by run-time support library (set of functions built into libraries included with compiler)

SYSTEM CALL PARAMETER PASSING

- ❑ Often, more information is required than simply identity of desired system call
- ❑ Exact type and amount of information vary according to OS and call
- ❑ Three general methods used to pass parameters to the OS
 - ❑ **Simplest:** pass the parameters in registers
 - ❑ In some cases, may be more parameters than registers
 - ❑ **Parameters stored in a block, or table, in memory,** and address of block passed as a parameter in a register. This approach taken by Linux and Solaris
 - ❑ **Parameters placed, or pushed, onto the stack by the program and popped off the stack by the operating system**
 - ❑ Block and stack methods do not limit the number or length of parameters being passed

PARAMETER PASSING VIA TABLE/BLOCK



SYSTEM CALLS – WORKING STEPS

- ❑ A user program wants to read data from a file:
 - ❑ The user program starts executing in the user mode.
 - ❑ The program issues a read system call to request the operating system to read data from the file.
 - ❑ The processor identifies the read system call and invokes it.
 - ❑ The control of the process goes to the kernel mode.
 - ❑ The operating system performs the necessary actions to read data from the file.
 - ❑ Once the read system call's execution gets complete, the control of the process returns to the user mode.
 - ❑ The user program continues executing with the data that was read from the file.

SYSTEM CALLS – TYPES

❑ Process control

- ❑ create process, terminate process
- ❑ end, abort
- ❑ load, execute
- ❑ get process attributes, set process attributes
- ❑ wait for time
- ❑ wait event, signal event
- ❑ allocate and free memory
- ❑ Dump memory if error
- ❑ Debugger for determining bugs, single step execution
- ❑ Locks for managing access to shared data between processes

SYSTEM CALLS – TYPES

❑ File management

- ❑ create file, delete file
- ❑ open, close file
- ❑ read, write, reposition
- ❑ get and set file attributes

❑ Device management

- ❑ Request device, release device
- ❑ Read, write, reposition
- ❑ Get device attributes, set device attributes
- ❑ Logically attach or detach devices

SYSTEM CALLS – TYPES

☐ **Information maintenance**

- ☐ get time or date, set time or date
- ☐ get system data, set system data
- ☐ get and set process, file, or device attributes

☐ **Communications**

- ☐ create, delete communication connection
- ☐ send, receive messages if message passing model to host name or process name
 - ☐ From client to server
- ☐ Shared-memory model create and gain access to memory regions
- ☐ transfer status information
- ☐ attach and detach remote devices

SYSTEM CALLS – TYPES

☐ Protection

- ☐ Control access to resources
- ☐ Get and set permissions
- ☐ Allow and deny user access

SYSTEM CALLS – EXAMPLE

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

PROTECTION: MODES – USER MODE

- ❑ The CPU is spending time in two very distinct modes: User Mode and Kernel Mode1.
- ❑ User Mode:
 - ❑ It is a restricted mode that limits the software's access to system resources.
 - ❑ Code running in user mode must delegate to system APIs to access hardware or memory.
 - ❑ When a user-mode application is started, Windows creates a process for the application.
 - ❑ The process provides the application with a private virtual address space and a private handle table. Because an application's virtual address space is private, one application can't alter data that belongs to another application. Each application runs in isolation, and if an application crashes, the crash is limited to that one application.

PROTECTION: MODES – KERNEL MODE

❑ Kernel Mode:

- ❑ It is a privileged mode that allows software to access system resources and perform privileged operations.
- ❑ The executing code has complete and unrestricted access to the underlying hardware.
- ❑ It can execute any CPU instruction and reference any memory address.
- ❑ All code that runs in kernel mode shares a single virtual address space.
 - ❑ Therefore, a kernel-mode driver isn't isolated from other drivers and the operating system itself. If a kernel-mode driver accidentally writes to the wrong virtual address, data that belongs to the operating system or another driver could be compromised. If a kernel-mode driver crashes, the entire operating system crashes.

INTERRUPTS

- ❑ An interrupt is a signal emitted by hardware or software when a process or an event needs immediate attention. It alerts the processor to a high-priority process requiring interruption of the current working process.
- ❑ When a device raises an interrupt at let's say process i, the processor first completes the execution of instruction i. Then it loads the Program Counter (PC) with the address of the first instruction of the ISR. Before loading the Program Counter with the address, the address of the interrupted instruction is moved to a temporary location.

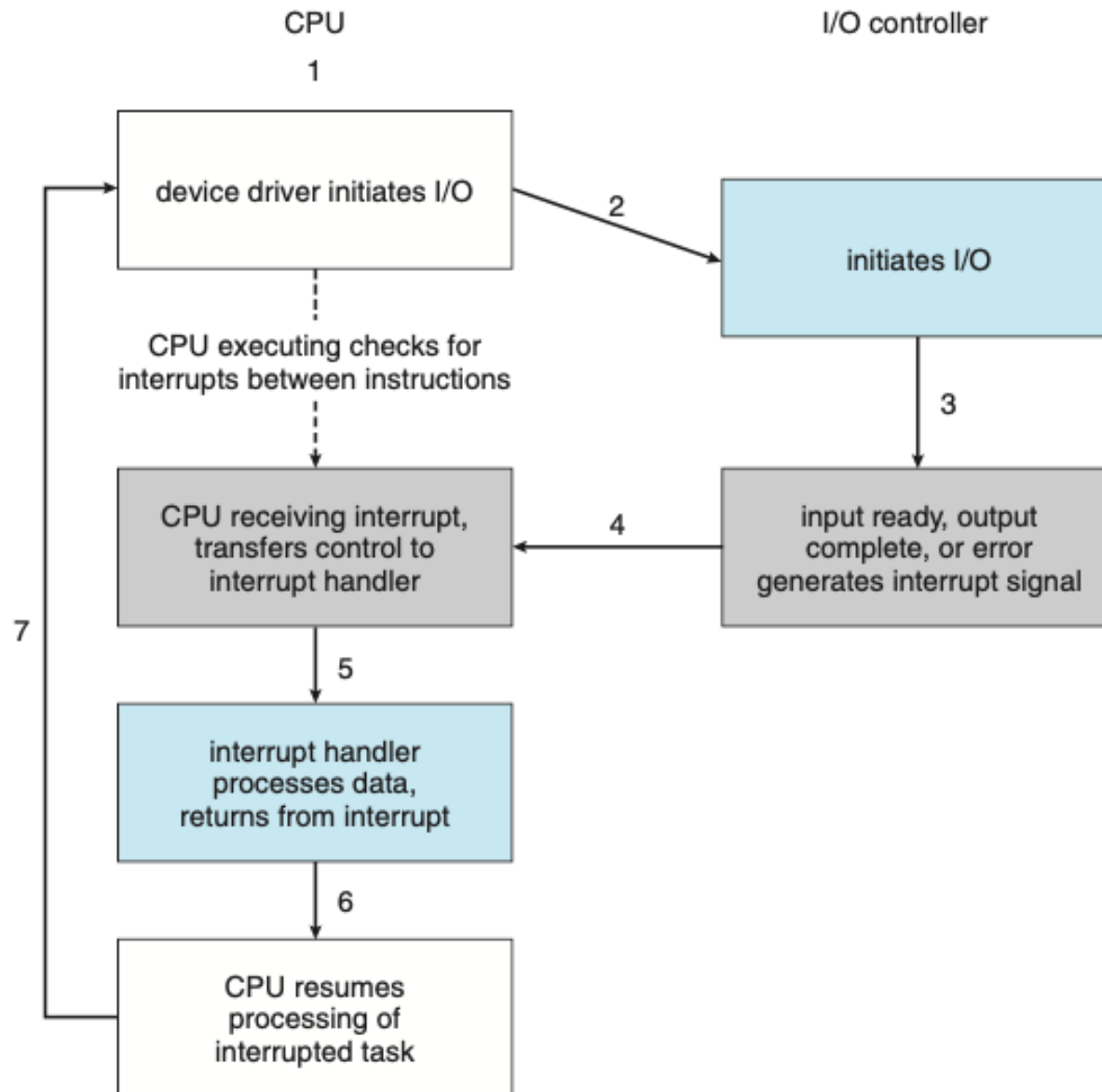
INTERRUPT IMPLEMENTATION

- ❑ The CPU hardware has a wire called the interrupt-request line that the CPU senses after executing every instruction.
- ❑ When the CPU detects that a controller has asserted a signal on the interrupt-request line, it reads the interrupt number and jumps to the interrupt-handler routine by using that interrupt number as an index into the interrupt vector.
- ❑ It then starts execution at the address associated with that index. The interrupt handler saves any state it will be changing during its operation,

INTERRUPT IMPLEMENTATION

- ❑ determines the cause of the interrupt, performs the necessary processing, performs a state restore, and executes a return from interrupt instruction to return the CPU to the execution state prior to the interrupt.
- ❑ the device controller raises an interrupt by asserting a signal on the interrupt request line, the CPU catches the interrupt and dispatches it to the interrupt handler, and the handler clears the interrupt by servicing the device.

INTERRUPT DRIVEN I/O CYCLE



PROCESS

- ❑ An operating system executes a variety of programs that run as a process.
- ❑ Process – a program in execution; process execution must progress in sequential fashion. No parallel execution of instructions of a single process
- ❑ Multiple parts
 - ❑ The program code, also called **text section**
 - ❑ Current activity including **program counter, processor registers**
 - ❑ **Stack** containing temporary data
 - ❑ **Function parameters, return addresses, local variables**
 - ❑ **Data section** containing global variables
 - ❑ **Heap** containing memory dynamically allocated during run time

PROCESS

- ❑ Program is passive entity stored on disk (executable file); process is active
 - ❑ Program becomes process when an executable file is loaded into memory.
- ❑ Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- ❑ One program can be several processes
 - ❑ Consider multiple users executing the same program

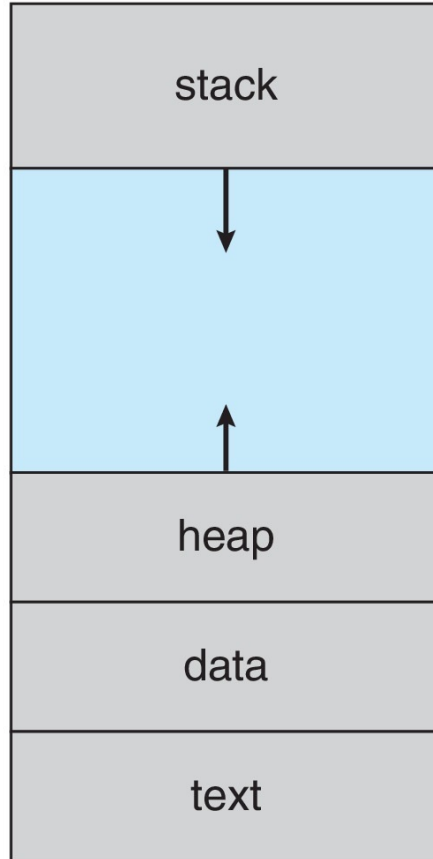
PROCESS – MEMORY LAYOUT

- ❑ Text section—the executable code
- ❑ Data section—global variables
- ❑ Heap section—memory that is dynamically allocated during program run time
- ❑ Stack section—temporary data storage when invoking functions (such as function parameters, return addresses, and local variables)
 - ❑ Each time a function is called, an **activation record** containing function parameters, local variables, and the return address is pushed onto the stack; when control is returned from the function, the activation record is popped from the stack.

PROCESS – MEMORY LAYOUT

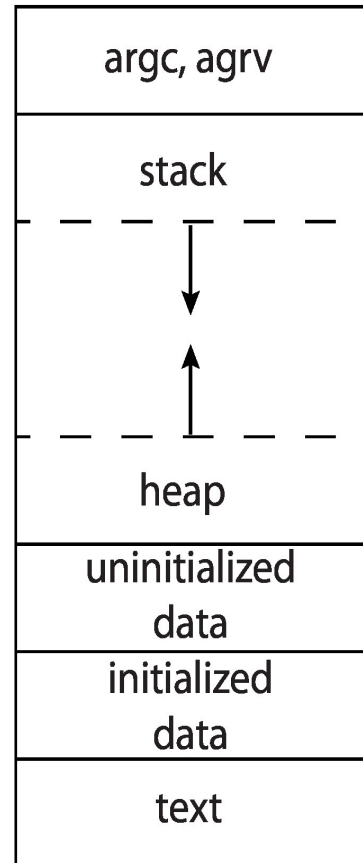


max



0

high
memory



low
memory

```
#include <stdio.h>
#include <stdlib.h>
```

```
int x;
int y = 15;
```

```
int main(int argc, char *argv[])
{
```

```
    int *values;
    int i;
```

```
    values = (int *)malloc(sizeof(int)*5);
```

```
    for(i = 0; i < 5; i++)
        values[i] = i;
```

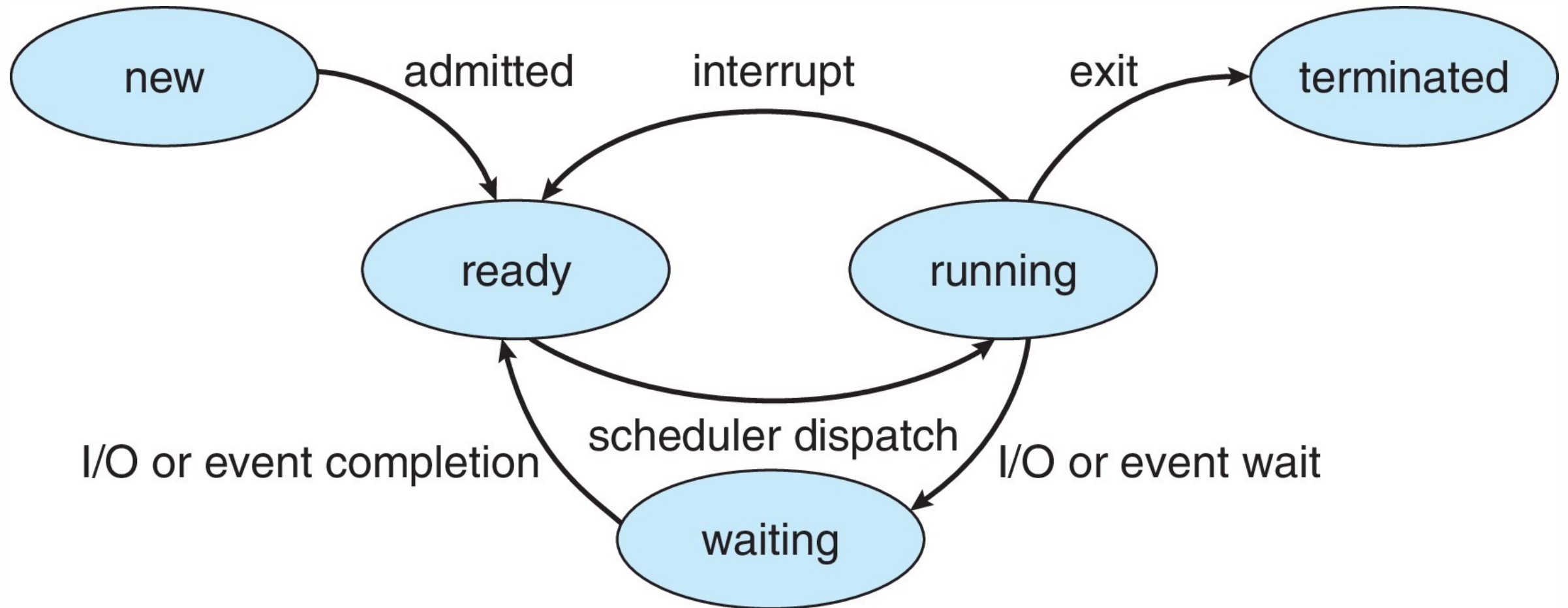
```
    return 0;
```

```
}
```

PROCESS – STATES

- ❑ As a process executes, it changes state. The state of a process is defined in part by the current activity of that process.
- ❑ A process may be in one of the following states:
 - ❑ **New**. The process is being created.
 - ❑ **Running**. Instructions are being executed.
 - ❑ **Waiting**. The process is waiting for some event to occur (such as an I/O completion or reception of a signal).
 - ❑ **Ready**. The process is waiting to be assigned to a processor.
 - ❑ **Terminated**. The process has finished execution.

PROCESS – STATES



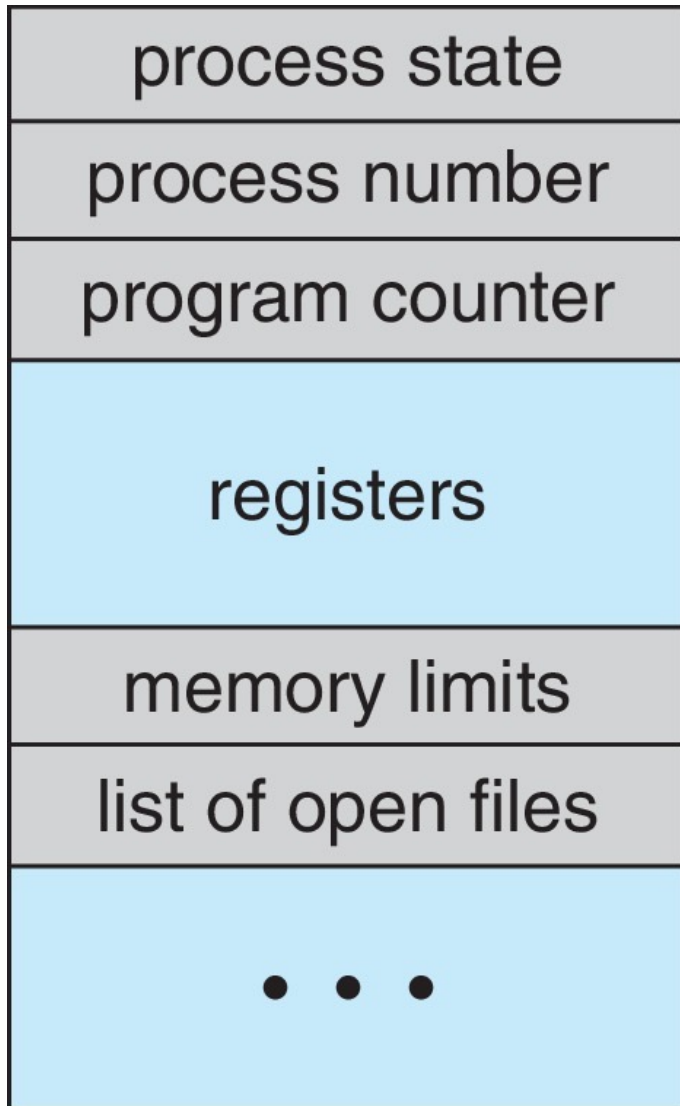
PROCESS – CONTROL BLOCK

- ❑ Each process is represented in the operating system by a process control block (PCB)—also called a task control block.
- ❑ Process state. The state may be new, ready, running, waiting, halted, and so on.
- ❑ Program counter. The counter indicates the address of the next instruction to be executed for this process.
- ❑ CPU registers. The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information. Along with the program counter, this state information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward when it is rescheduled to run.

PROCESS – CONTROL BLOCK

- ❑ CPU-scheduling information. This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.
- ❑ Memory-management information. This information may include such items as the value of the base and limit registers and the page tables, or the segment tables, depending on the memory system used by the operating system.
- ❑ Accounting information. This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
- ❑ I/O status information. This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

PROCESS – CONTROL BLOCK



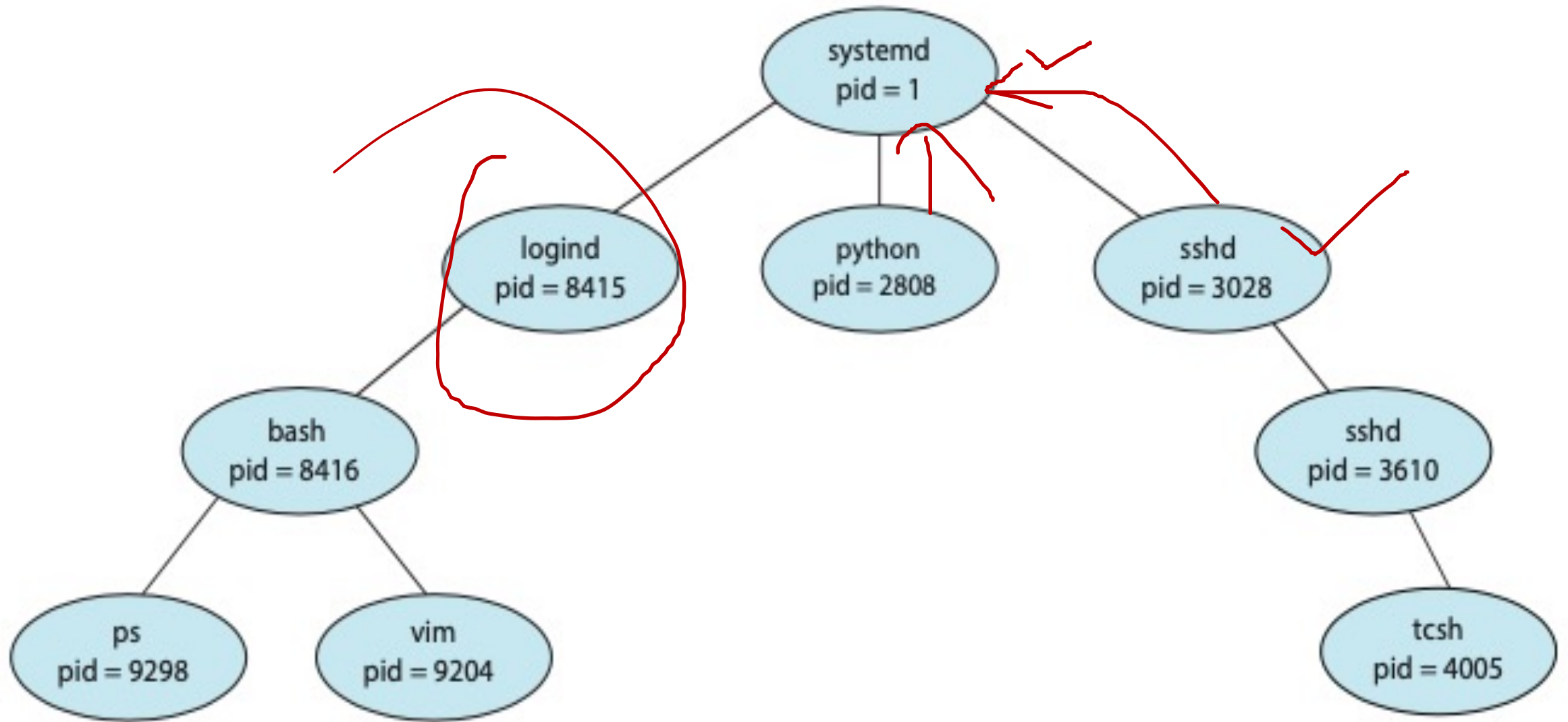
PROCESS – CREATION

- ❑ Process creation is the act of creating a new process in an operating system.
- ❑ A process can create several new processes through creating process system calls during the process execution.
- ❑ The creating process is called a **parent process**, and the new processes are called the **children** of that process.
- ❑ Each of these new processes may in turn create other processes, forming a **tree of processes**.

PROCESS – CREATION

- ❑ identify processes according to a **unique process identifier** (or pid), which is typically an integer number.
- ❑ The pid provides a unique value for each process in the system, and it can be used as an index to access various attributes of a process within the kernel.

PROCESS – CREATION



PROCESS – CREATION – PARENT & CHILD PROCESS

- ❑ When a process creates a child process, that child process will need certain resources (CPU time, memory, files, I/O devices) to accomplish its task.
 - ❑ A child process may be able to obtain its resources directly from the operating system, or it may be constrained to a subset of the resources of the parent process.
 - ❑ The parent may have to partition its resources among its children, or it may be able to share some resources (such as memory or files) among several of its children.
 - ❑ Parent & Child process share no resources.

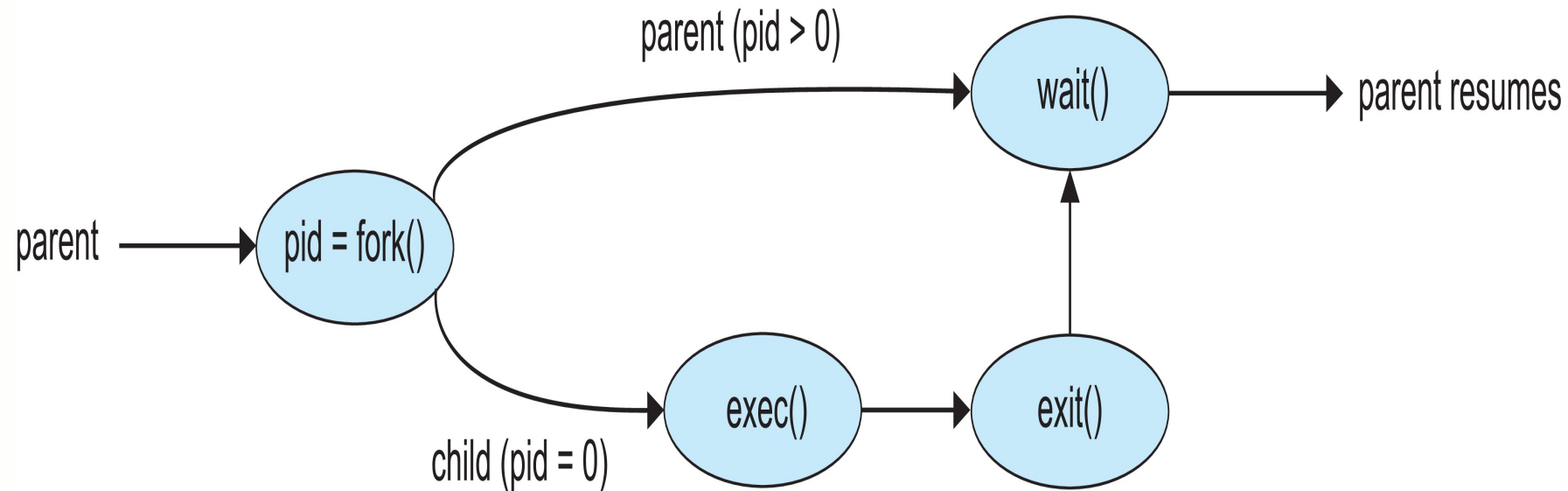
PROCESS – CREATION – PARENT & CHILD PROCESS

- ❑ When a process creates a new process, two possibilities for execution exist:
 - ❑ The parent continues to execute concurrently with its children.
 - ❑ The parent waits until some or all its children have terminated.

- ❑ There are also two address-space possibilities for the new process:
 - ❑ The child process is a duplicate of the parent process (it has the same program and data as the parent).
 - ❑ The child process has a new program loaded into it.

PROCESS – CREATION – FORK()

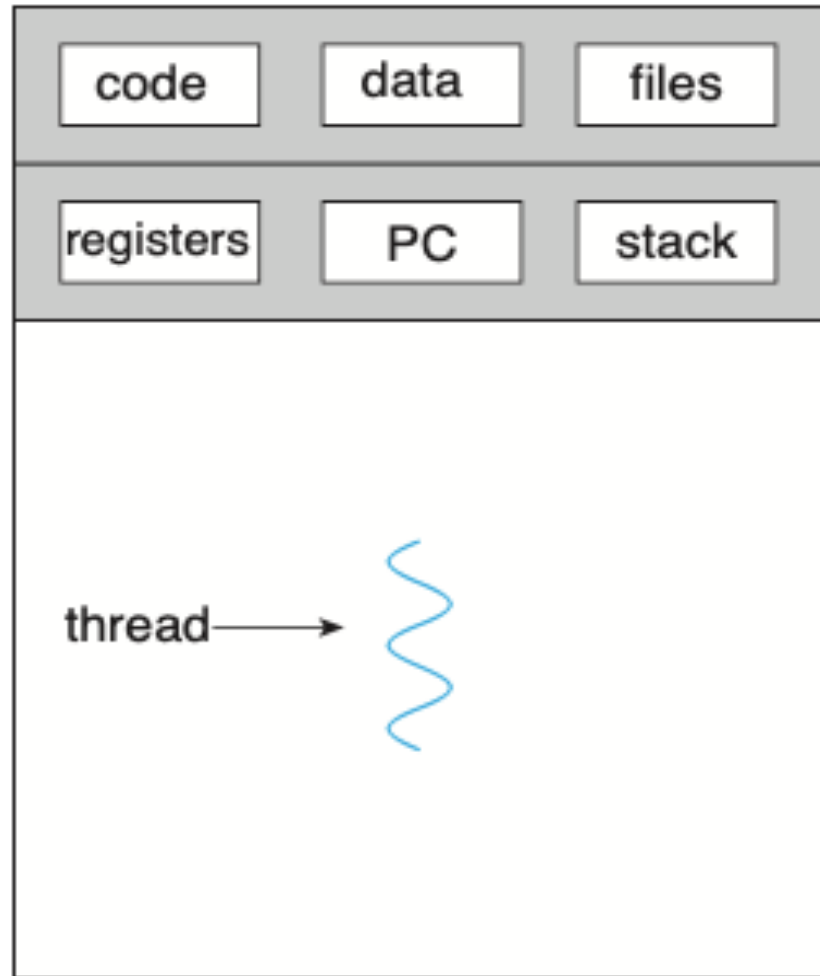
- ❑ `fork()` system call creates new process
- ❑ `exec()` system call used after a `fork()` to replace the process' memory space with a new program
- ❑ Parent process calls `wait()` waiting for the child to terminate



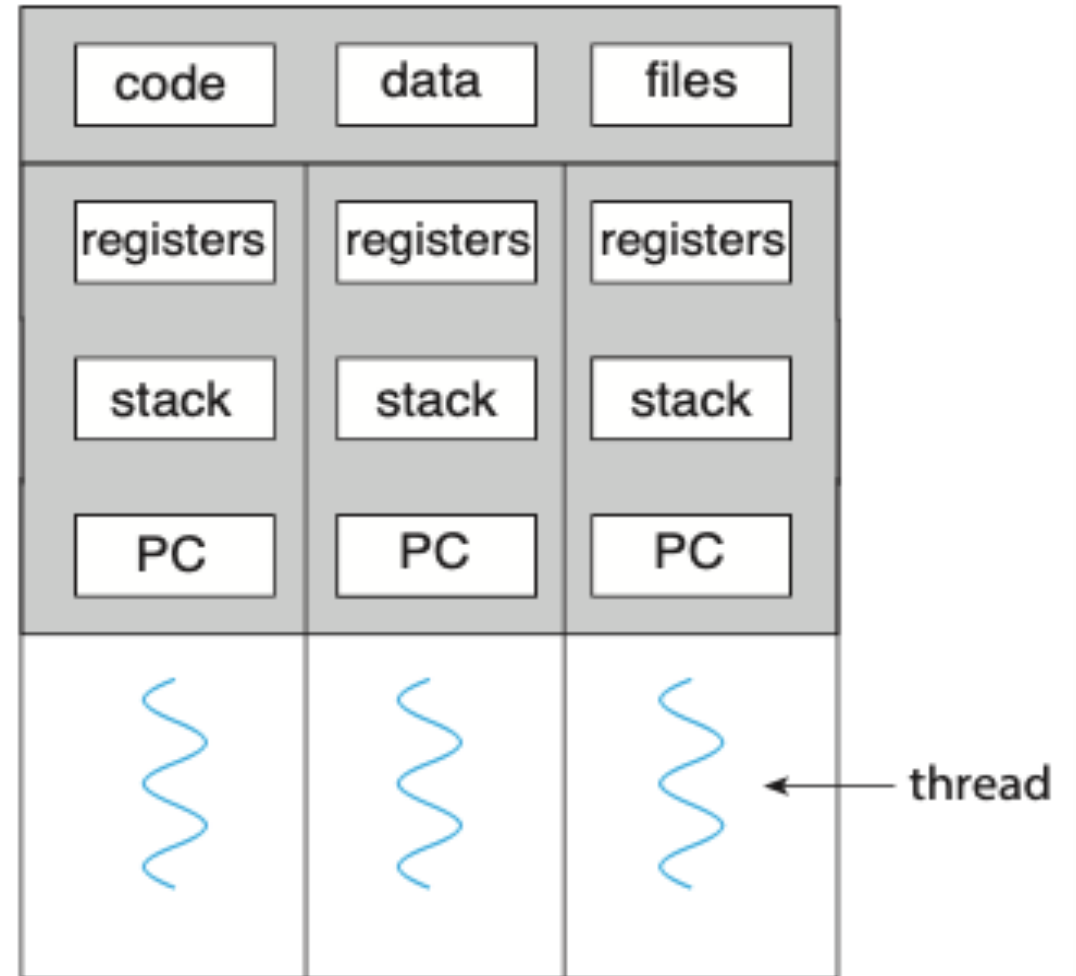
THREADS

- ❑ A thread is a basic unit of CPU utilization;
- ❑ It comprises a thread ID, a program counter (PC), a register set, and a stack. code section, data section, and other operating-system resources, such as open files and signals are **shared** among the **threads of same process**.
 - ❑ **Program counter**: It is a register in the CPU that holds the address of the next instruction to be executed.
 - ❑ **Register set**: It is a set of registers in the CPU that are used to store temporary data.
 - ❑ **Stack space**: It is a memory space used by the thread to store local variables and function calls.
- ❑ A traditional process has a single thread of control. If a process has multiple threads of control, it can perform more than one task at a time

THREADS - TYPES



single-threaded process

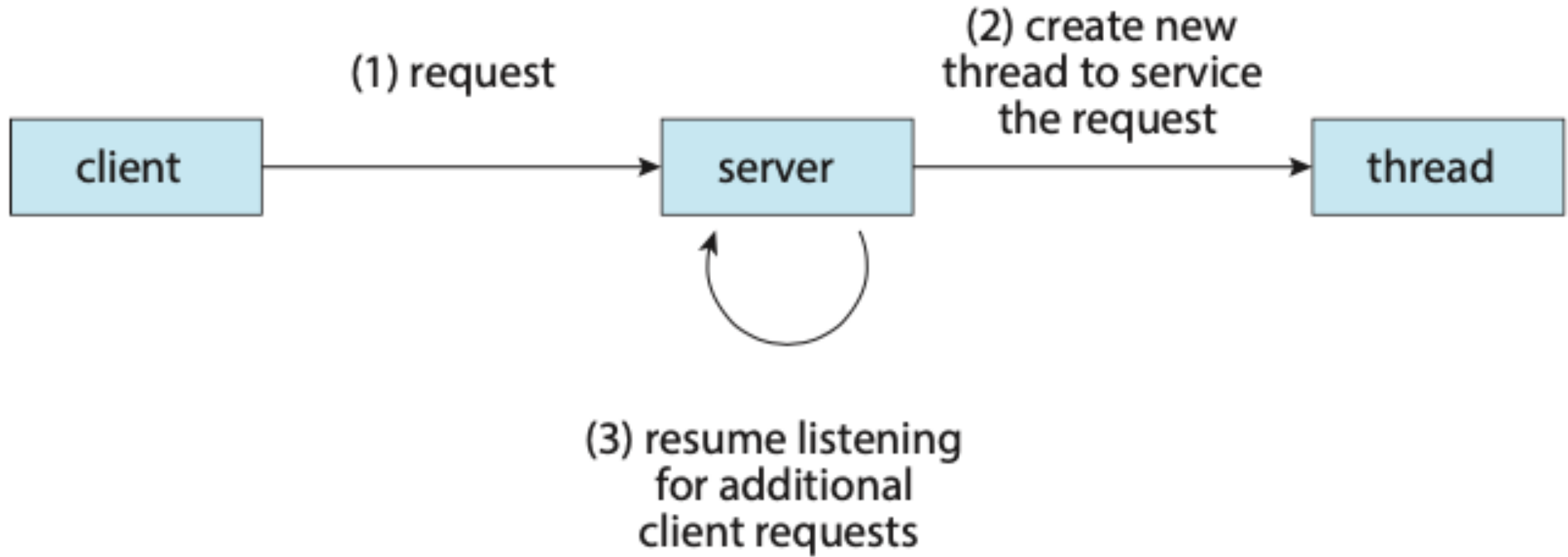


multithreaded process

THREADS - TYPES

- ❑ A **multithreaded process** is a system in which multiple threads are created of a process for increasing the computing speed of the system. In multithreading, many threads of a process are executed simultaneously.
- ❑ A **single-threaded process** is a process that can perform only one task at a time. It contains the execution of instructions in a single sequence, meaning one command is processed at a time

THREADS – EXAMPLE – MULTITHREADED ARCHITECTURE



THREADS – EXAMPLE – MULTITHREADED ARCHITECTURE

❑ Benefits

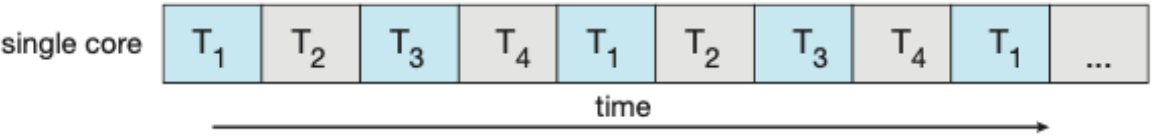
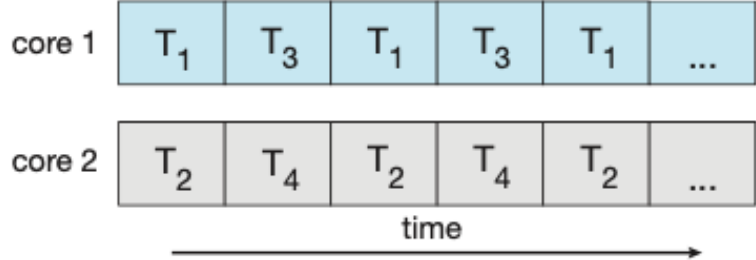
- ❑ **Responsiveness.** Multithreading an interactive application may allow a program to continue running even if part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user.
- ❑ **Resource sharing.** Processes can share resources only through techniques such as shared memory and message passing. Threads share the memory and the resources of the process to which they belong by default.
- ❑ **Economy.** Allocating memory and resources for process creation is costly. Because threads share the resources of the process to which they belong, it is more economical to create and context-switch threads.
- ❑ **Scalability.** The benefits of multithreading can be even greater in a multiprocessor architecture, where threads may be running in parallel on different processing cores.

THREADS – MULTICORE PROGRAMMING

- ❑ Place multiple computing cores on a single processing chip where each core appears as a separate CPU to the operating system – Multicore Systems.
- ❑ Multithreaded programming provides a mechanism for more efficient use of these multiple computing cores and improved concurrency.

THREADS – MULTICORE – CONCURRENCY VS PARALLELISM

❑ Consider an application with four threads.

Single core System	Multicore System
Concurrency merely means that the execution of the threads will be interleaved over time because the processing core can execute only one thread at a time.	Concurrency means that some threads can run in parallel, because the system can assign a separate thread to each core.
 The diagram shows a single core executing four threads (T1, T2, T3, T4) in an interleaved manner over time. The threads are represented as blocks in a sequence: T1, T2, T3, T4, T1, T2, T3, T4, T1, ... with a time axis below.	 The diagram shows a multicore system with two cores. Core 1 executes threads T1 and T3 in parallel, while Core 2 executes threads T2 and T4 in parallel. The threads are represented as blocks in two parallel sequences: Core 1 (T1, T3, T1, T3, T1, ...) and Core 2 (T2, T4, T2, T4, T2, ...) with a time axis below.
A concurrent system supports more than one task by allowing all the tasks to make progress. .(concurrency without parallelism)	A parallel system can perform more than one task simultaneously.

THREADS – MULTICORE – CONCURRENCY VS PARALLELISM

❑ Challenges:

- ❑ To modify existing programs as well as design new programs that are multithreaded.
- ❑ **Identifying tasks.** This involves examining applications to find areas that can be divided into separate, concurrent tasks. Ideally, tasks are independent of one another and thus can run in parallel on individual cores.
- ❑ **Balance.** While identifying tasks that can run in parallel, programmers must also ensure that the tasks perform equal work of equal value.
- ❑ **Data splitting.** Just as applications are divided into separate tasks, the data accessed and manipulated by the tasks must be divided to run on separate cores.
- ❑ **Data dependency.** The data accessed by the tasks must be examined for dependencies between two or more tasks. When one task depends on data from another, programmers must ensure that the execution of the tasks is synchronized to accommodate the data dependency.

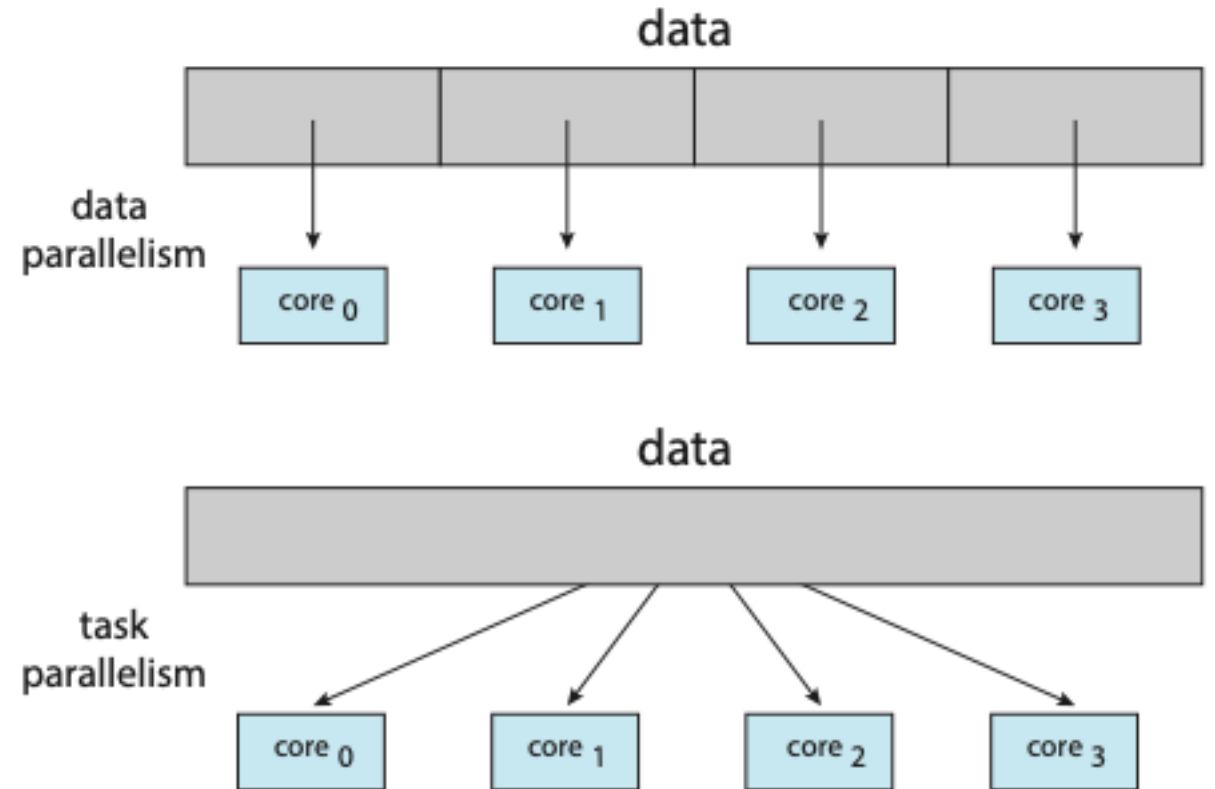
THREADS – MULTICORE – CONCURRENCY VS PARALLELISM

❑ Challenges:

- ❑ **Testing and debugging.** When a program is running in parallel on multiple cores, many different execution paths are possible. Testing and debugging such concurrent programs is inherently more difficult than testing and debugging single-threaded applications.

THREADS – TYPES OF PARALLELISM

- ❑ **Data parallelism** focuses on distributing subsets of the same data across multiple computing cores and performing the same operation on each core.
- ❑ **Task parallelism** involves distributing not data but tasks (threads) across multiple computing cores. Each thread is performing a unique operation. Different threads may be operating on the same data, or they may be operating on different data.



THREADS – TYPES OF PARALLELISM

❑ Data parallelism:

- ❑ when each processor performs the same task on different distributed data.
- ❑ It focuses on distributing the data across different nodes, which operate on the data in parallel.

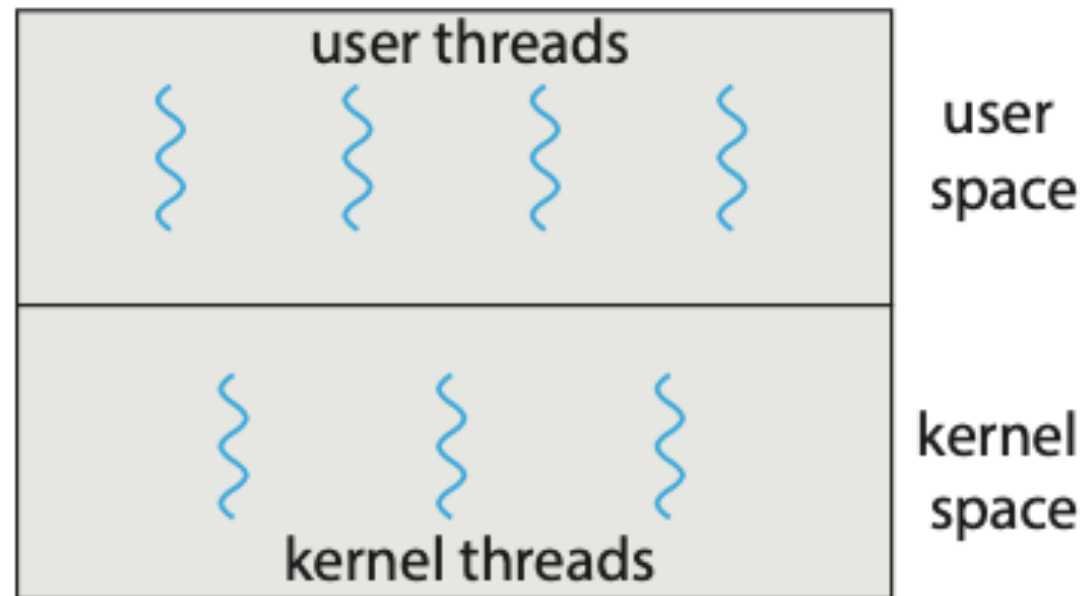
THREADS – TYPES OF PARALLELISM

❑ Task parallelism:

- ❑ It means concurrent execution of the different task on multiple computing cores
- ❑ when each processor executes a different thread (or process) on the same or different data.
- ❑ The threads may execute the same or different code.

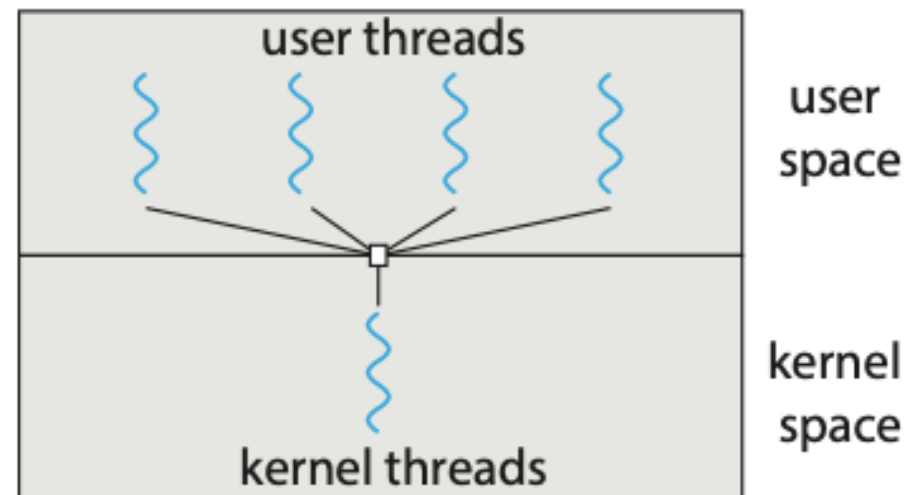
THREADS – MULTITHREADING MODEL

- ❑ User threads are supported above the kernel and are managed without kernel support, whereas kernel threads are supported and managed directly by the operating system.



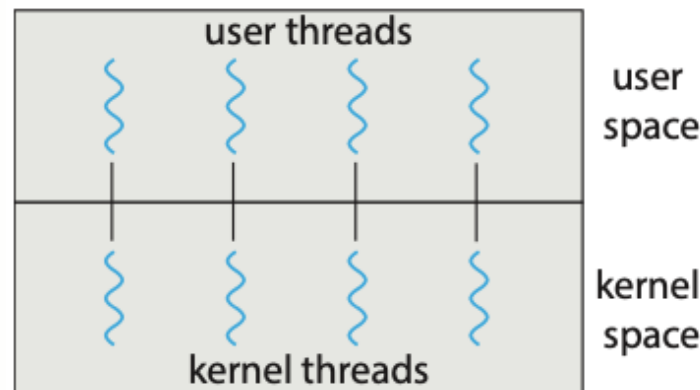
THREADS – MULTITHREADING MODEL – MANY TO ONE

- ❑ Maps many user-level threads to one kernel thread.
- ❑ Thread management is done by the thread library in user space.
- ❑ The entire process will block if a thread makes a blocking system call.
- ❑ Only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multicore systems.



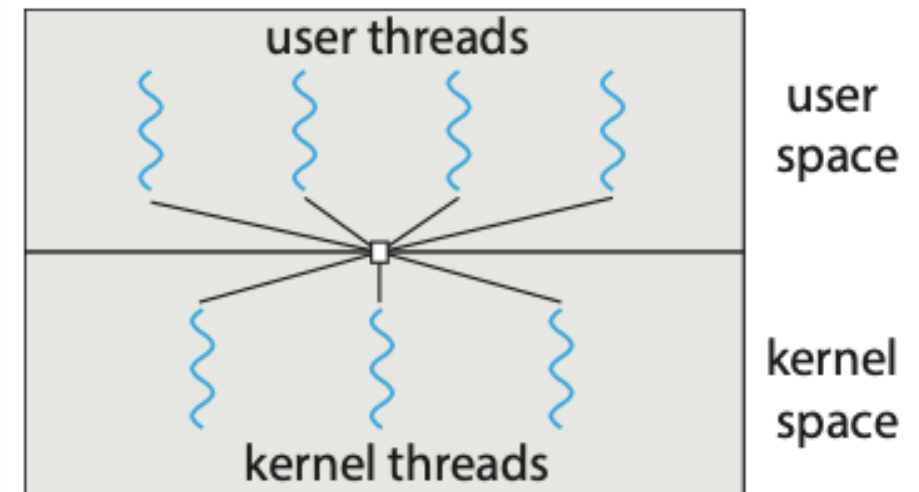
THREADS – MULTITHREADING MODEL – ONE TO ONE

- ❑ Maps each user thread to a kernel thread.
- ❑ It provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call.
- ❑ allows multiple threads to run in parallel on multiprocessors.
- ❑ Creating a user thread requires creating the corresponding kernel thread, and a large number of kernel threads may burden the performance of a system.



THREADS – MULTITHREADING MODEL – MANY TO MANY

- ❑ Multiplexes many user-level threads to a smaller or equal number of kernel threads.
- ❑ developers can create as many user threads as necessary, and the corresponding kernel threads can run in parallel on a multiprocessor.
- ❑ when a thread performs a blocking system call, the kernel can schedule another thread for execution.



THREADS – MULTITHREADING MODEL – MANY TO MANY

❑ Why One to Many Model Do not Exist? Some reasons

- ❑ It maps many user-level threads to one kernel-level thread.
- ❑ This model does not allow individual processes to be split across multiple CPUs because a single kernel thread can operate only on a single CPU.
- ❑ When a thread makes a blocking system call, the entire process will be blocked.
- ❑ Only one thread can access the kernel at a time, so multiple threads are unable to run in parallel on multiprocessors.
- ❑ lead to poor performance and scalability issues.